NoSQL Databases

Ehsan Khamespanah

Spring 2024

Introduction to Database Transactions

- Lets start by an example
  - Lost updates
  - Inconsistent read

Thread 1: A = A + A * 0.1

Thread 2: A = A - 10

Scenario 1:

$Read_1(A)$: A = 100
$Write_1(A)$: A = 110
$Read_2(A)$: A = 110
$Write_2(A)$: A = 100

Scenario 2:

$Read_2(A)$: A = 100
$Write_2(A)$: A = 90
$Read_1(A)$: A = 90
$Write_1(A)$: A = 99

Scenario 3:

$Read_1(A)$: A = 100
$Read_2(A)$: A = 100
$Write_1(A)$: A = 110
$Write_2(A)$: A = 90

Transactions

- A bounded sequence of work
- All resources are in consistent state
  - When the transaction begins and when it ends
- Must complete on an all-or-nothing basis
- ACID properties
  - **A**tomicity: the operation cannot split down into its individual processes. All of the actions that take place inside it pass or fail at the same time. If any individual operation fails, the entire transaction has been deemed a failure and must be reversed (rolled back)

Transactions

- **C**onsistency: Guaranteeing that perhaps the data updated by the transaction conforms with all of the column requirements, ensuring data security
- **I**solation: Each transaction is kept separate from the others, results in having no impact on transactions that are executed simultaneously
- **D**urability: Data alterations that occur inside a completed transaction may be securely regarded to be saved in the system irrespective of what else might follow, which helps with durability.

Database Transactions

- Designed for all purposes ACID
- Strong consistency, concurrency, recovery
- Mathematical background
- Standard Query language (SQL)
- Lots of tools to use with i.e: Reporting services, entity frameworks, ...
- Vertical scaling (up scaling)

Relaxing Isolation for Better Performance

- SQL defines a set of reduced isolation levels

Isolation Level	Dirty Read	Unrepeatable Read	Phantom
Read Uncommitted	Yes	Yes	Yes
Read Committed	No	Yes	Yes
Repeatable Read	No	No	Yes
Serializable	No	No	No

Transaction in Practice

- What we have in JDBC

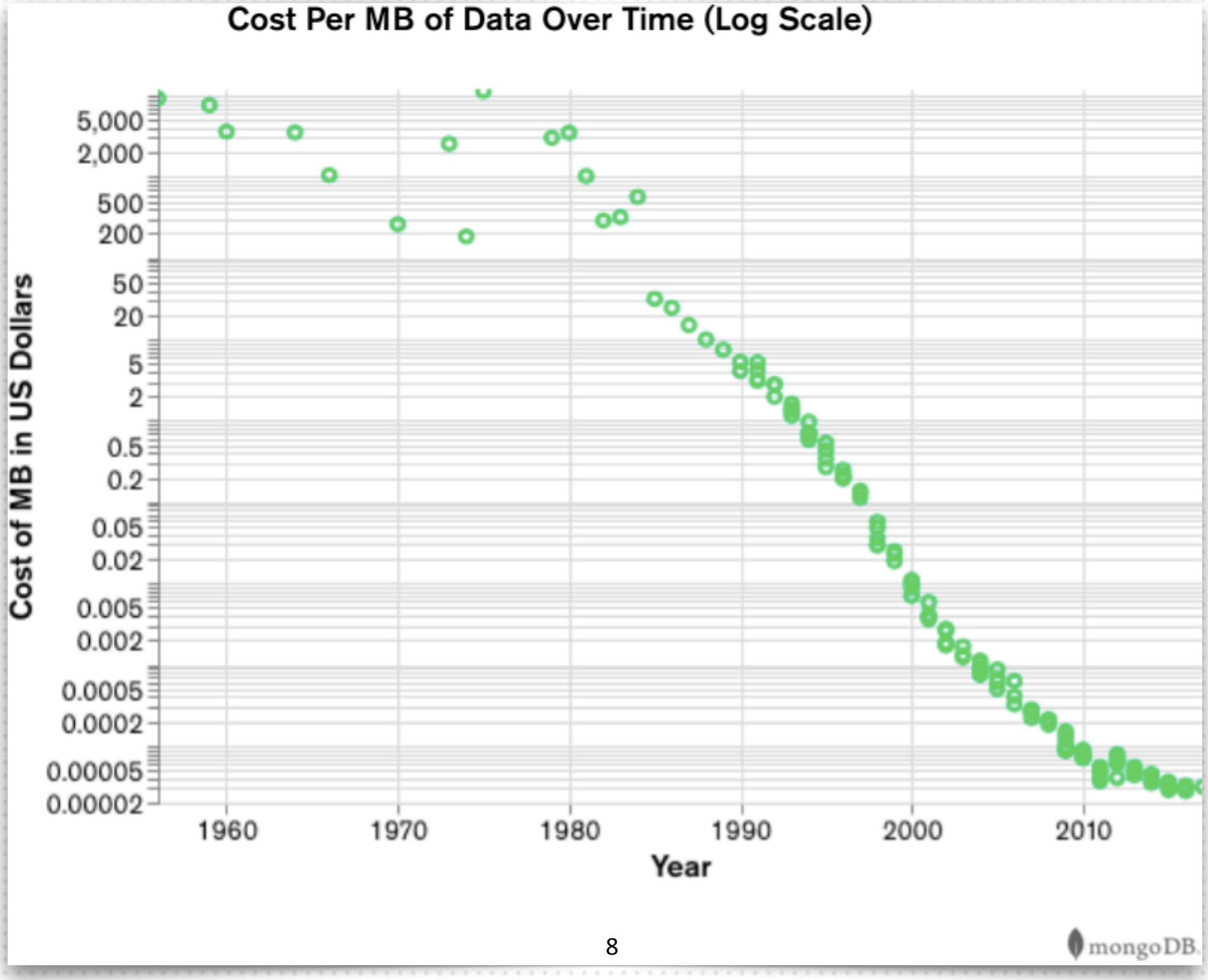
```
con.setAutoCommit(boolean);
//database updates
con.commit();
con.rollback();
```

- In JPA

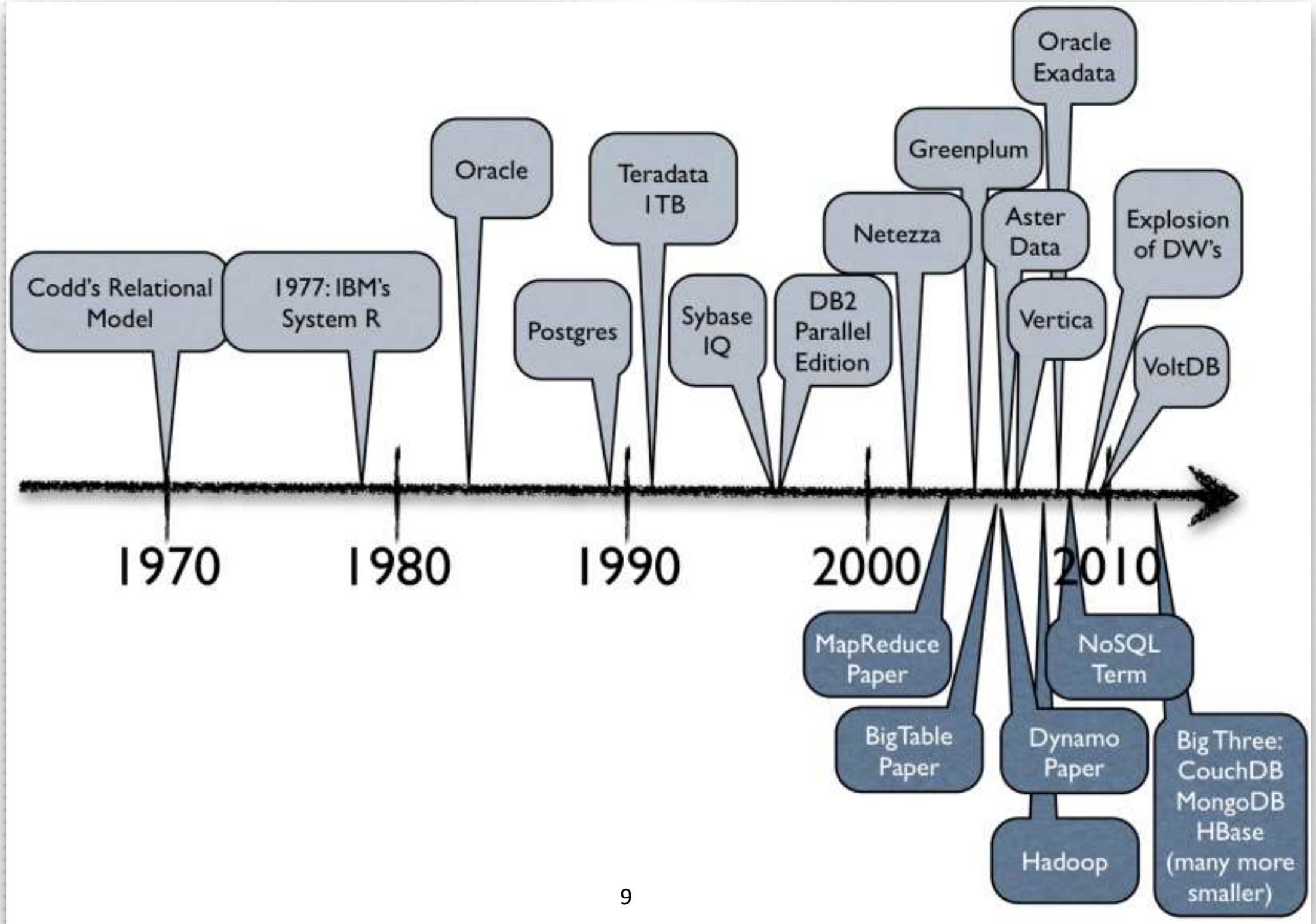
```
entityManager.getTransaction().begin();
//database updates
entityManager.getTransaction().commit();
entityManager.getTransaction().rollback();
```

- Stored Procedures

A Brief History of Databases

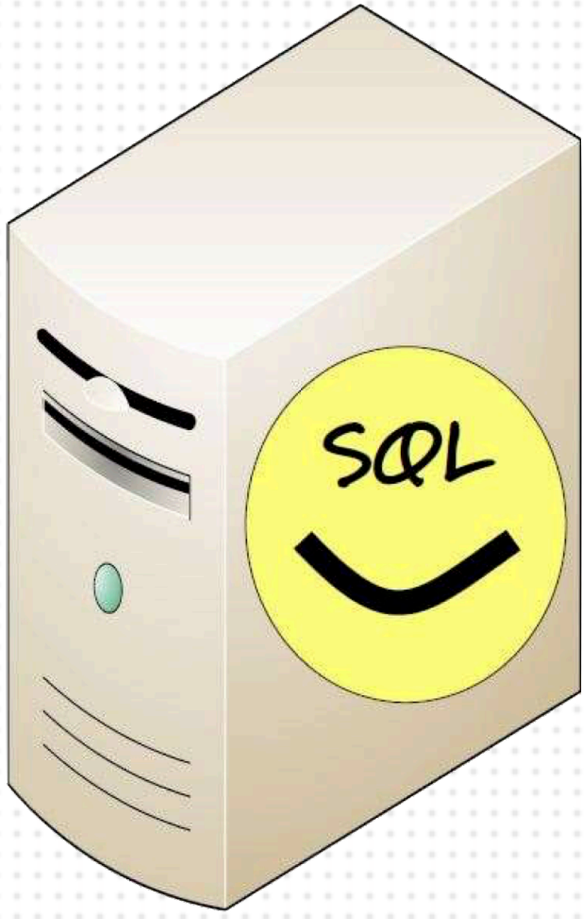


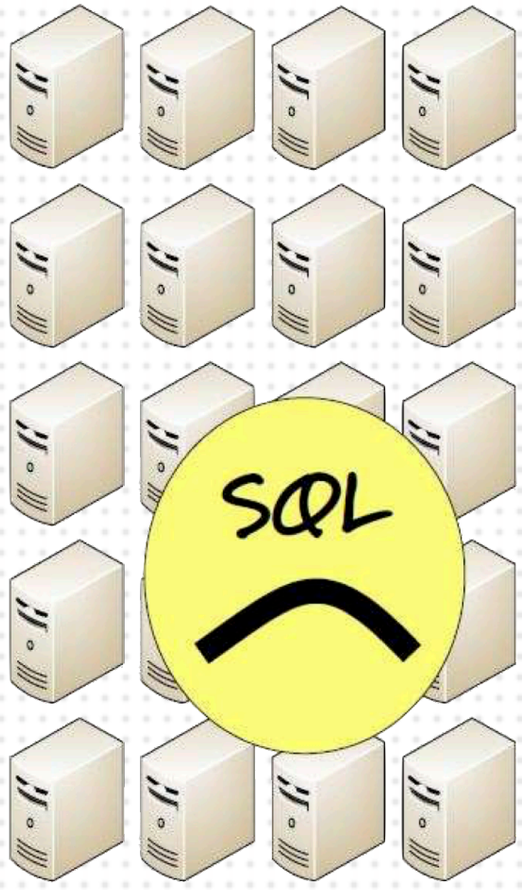
A Brief History of Databases



Era of Distributed Computing

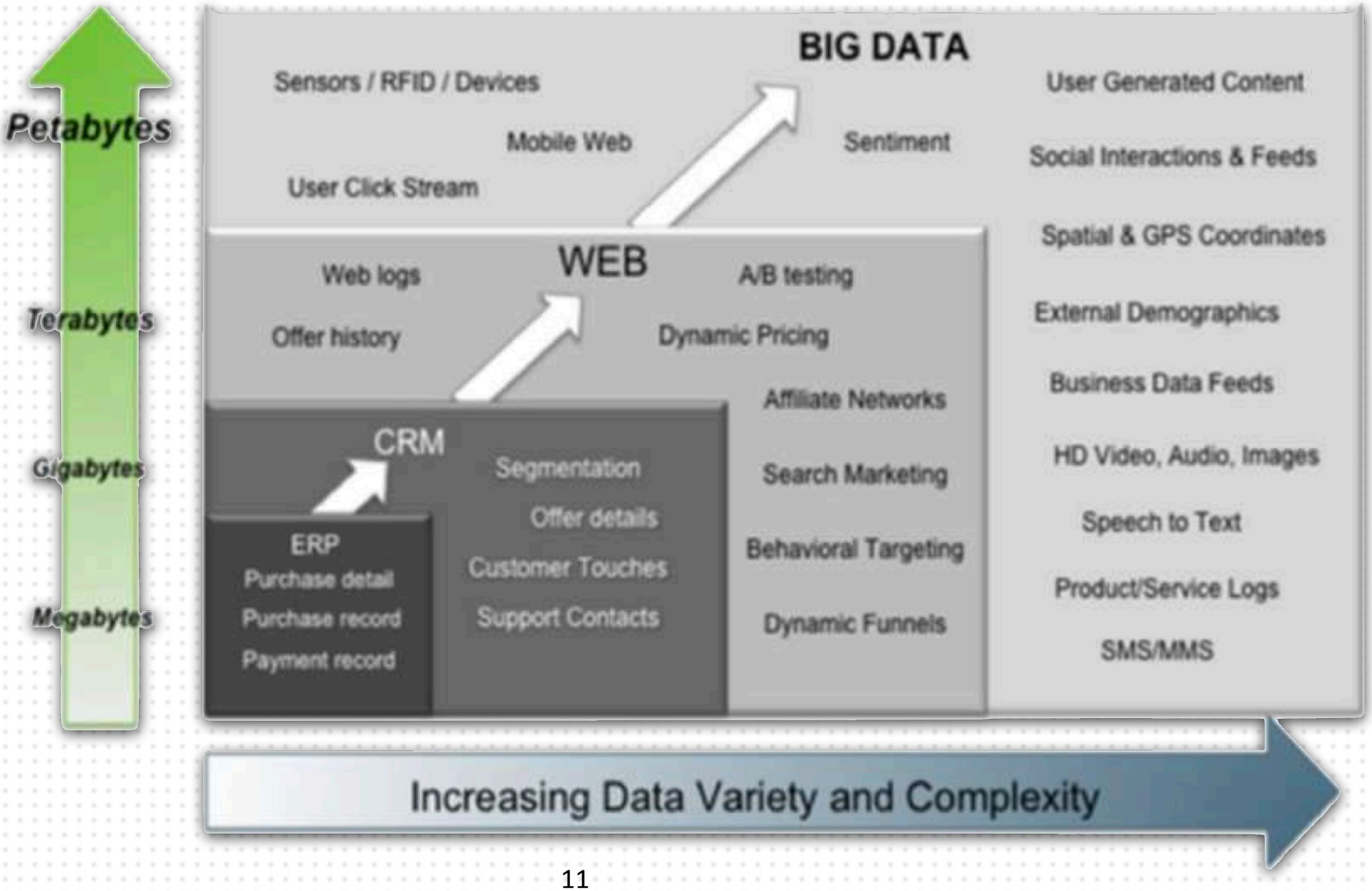
- Relational databases were not built for distributed applications.
- Because…
  - Joins are expensive
  - Hard to scale horizontally
- Impedance mismatch occurs
- Expensive (product cost, hardware, maintenance)



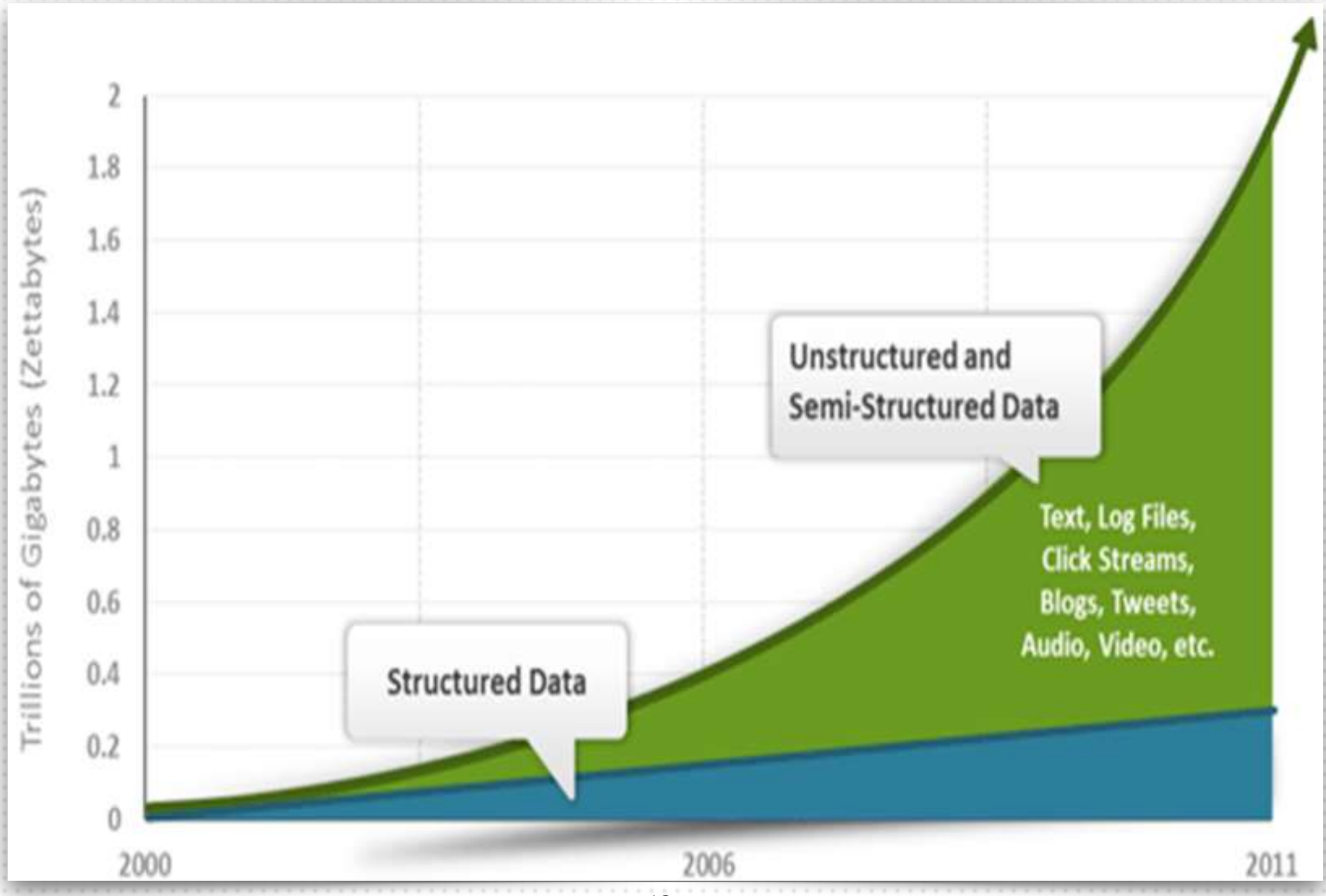


Rise of Big Data

- Three V of big data: Volume, Velocity, Verity



Rise of Big Data

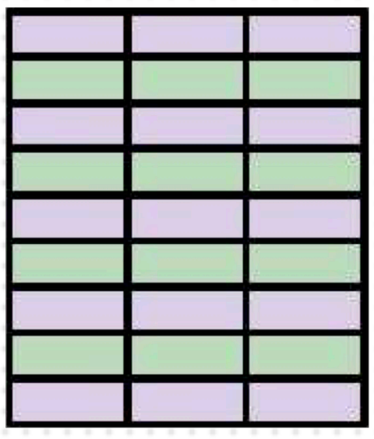


Rise of Big Data

- Walmart: 1 million transactions per hour
- Facebook: 40 billion photos
- People are talking about petabytes today

Data Sources

were



will be

text

image

video

connections

NoSQL why, what and when?

- Google & Amazon built their own databases (Big table & Dynamo) Facebook invented Cassandra and is using thousands of them
- #NoSQL was a twitter hashtag for a conference in 2009
- The name doesn’t indicate its characteristics
- There is no strict definition for NoSQL databases
- There are more than 150 NoSQL databases (nosql-database.org)



Characteristics of NoSQL Databases

- Non relational
- Cluster friendly
- Schema-less
- 21 century web
- Open-source



Characteristics of NoSQL Databases

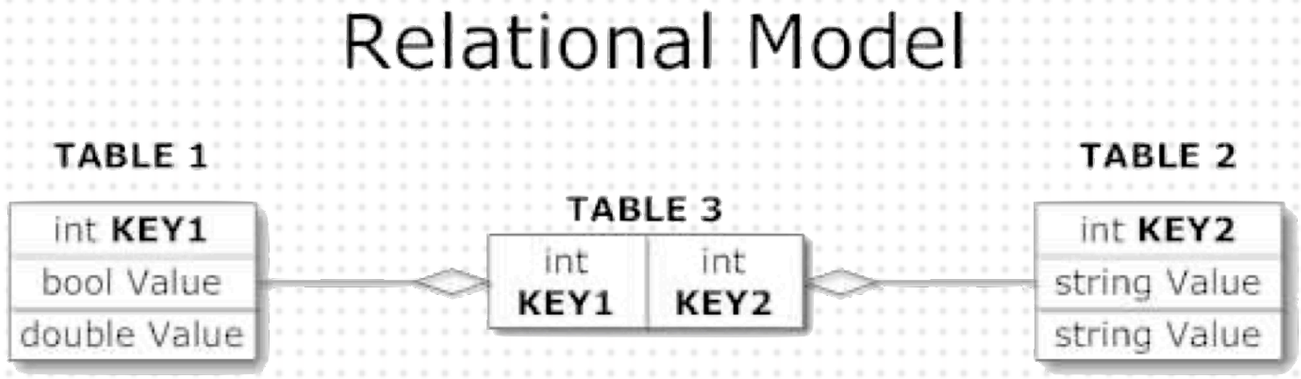
- NoSQL avoids:
  - Overhead of ACID transactions
  - Complexity of SQL query
  - Burden of up-front schema design
  - DBA presence
  - Transactions (It should be handled at application layer)
- Provides:
  - Easy and frequent changes to DB
  - Horizontal scaling (scaling out)
  - Solution to Impedance mismatch
  - Fast development

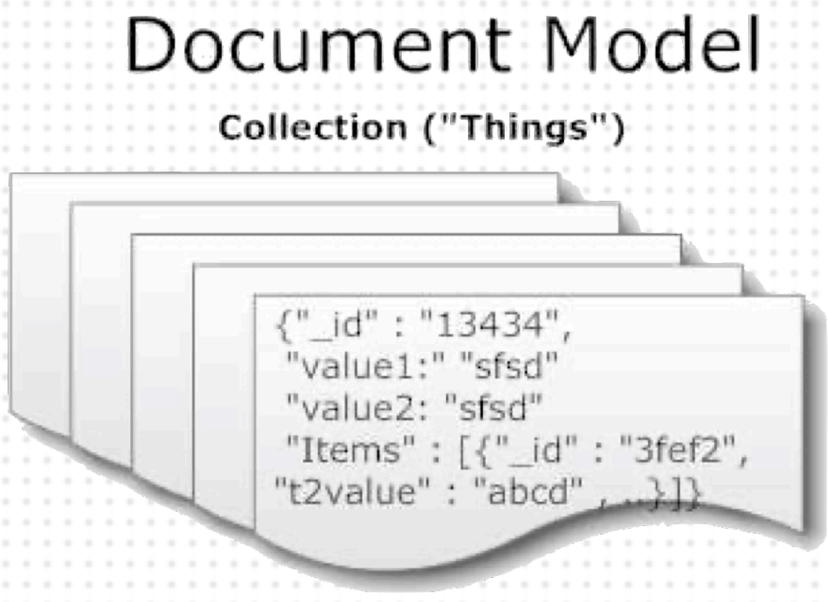
Schema-Based Data Model

- You can’t add a record which does not fit the schema
- You need to add NULLs to unused items in a row
- We should consider the datatypes. i.e : you can’t add a string to an integer field
- You can’t add multiple items in a field (You should create another table: primary-key, foreign key, joins, normalization, ... !!!)

There is No … in NoSQL Databases

- No schema to consider
- No unused cell
- No datatype (implicit)
- Most of considerations are done in application layer
- Items are gathered in an aggregate (document)

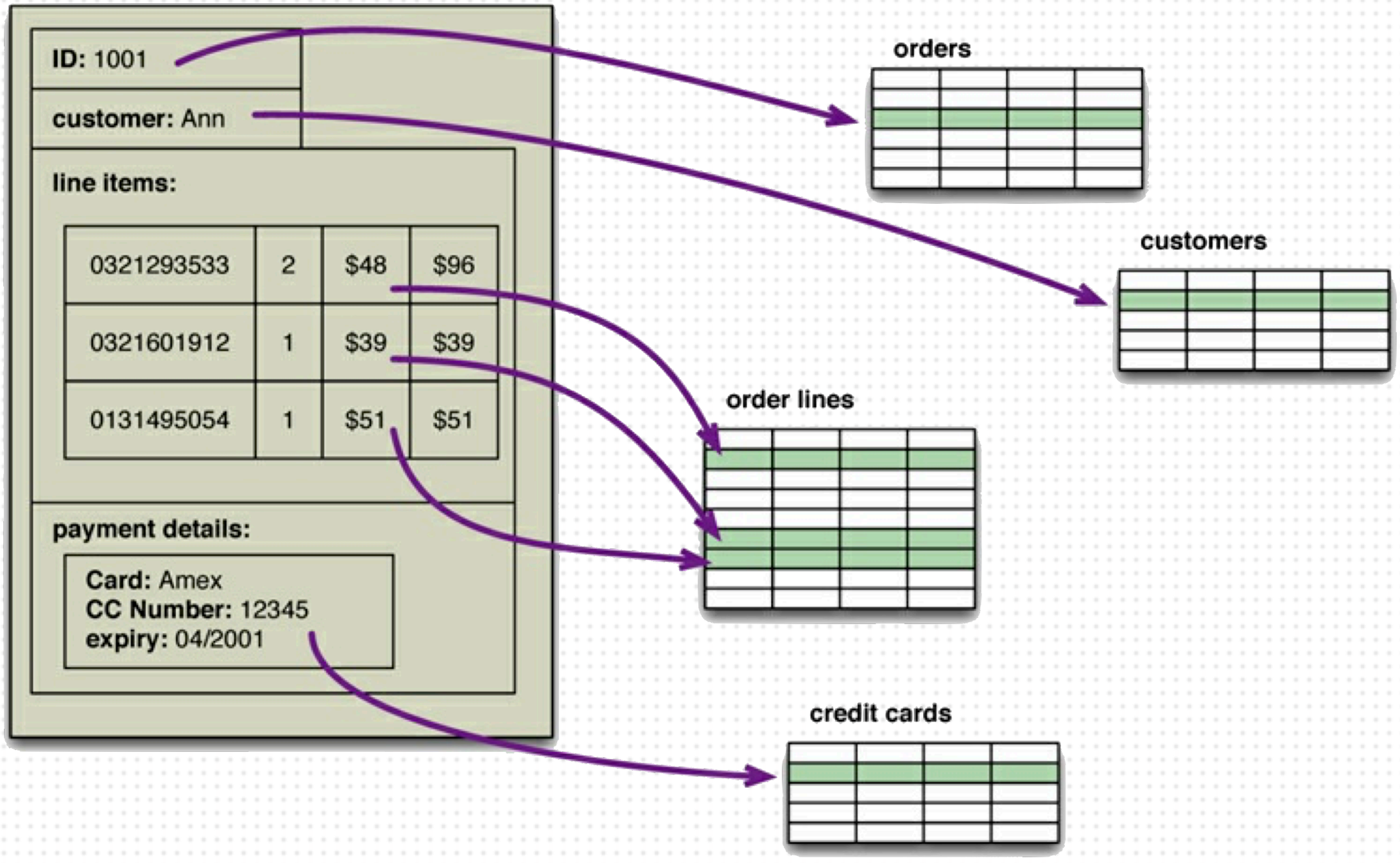




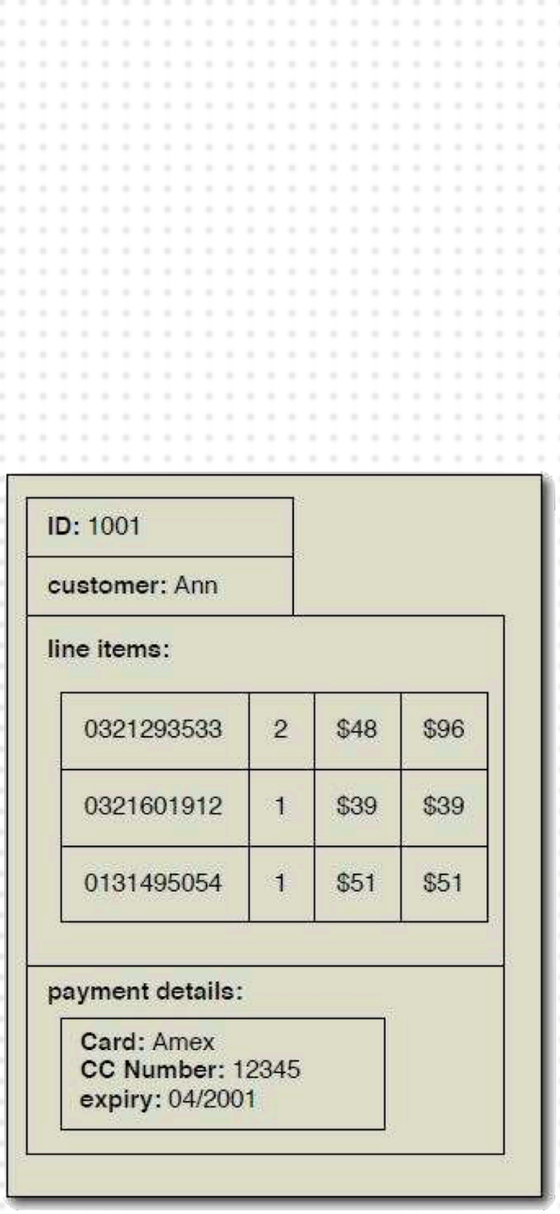
What is Aggregation?

- The term comes from Domain Driven Design
- Shared nothing architecture
- An aggregate is a cluster of domain objects that can be treated as a single unit
- Aggregates are the basic element of transfer of data storage - you request to load or save whole aggregates
- Transactions should not cross aggregate boundaries
- This mechanism reduces the join operations to a minimal level

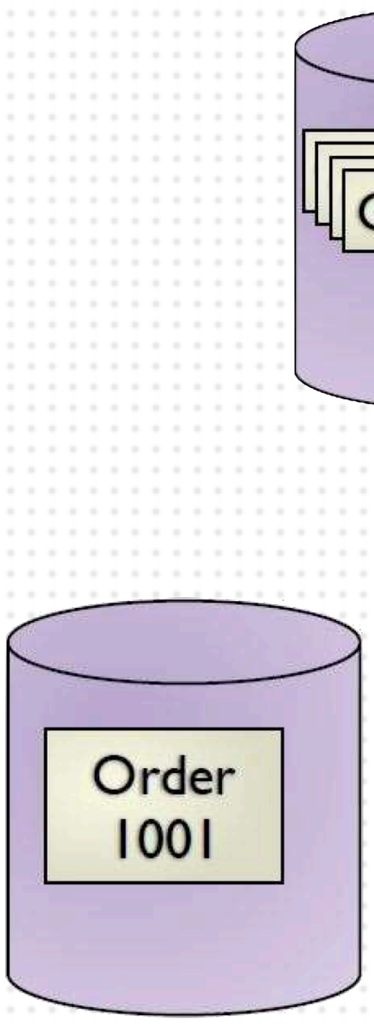
An Example of Aggregation



An Example of Aggregation



aggregate

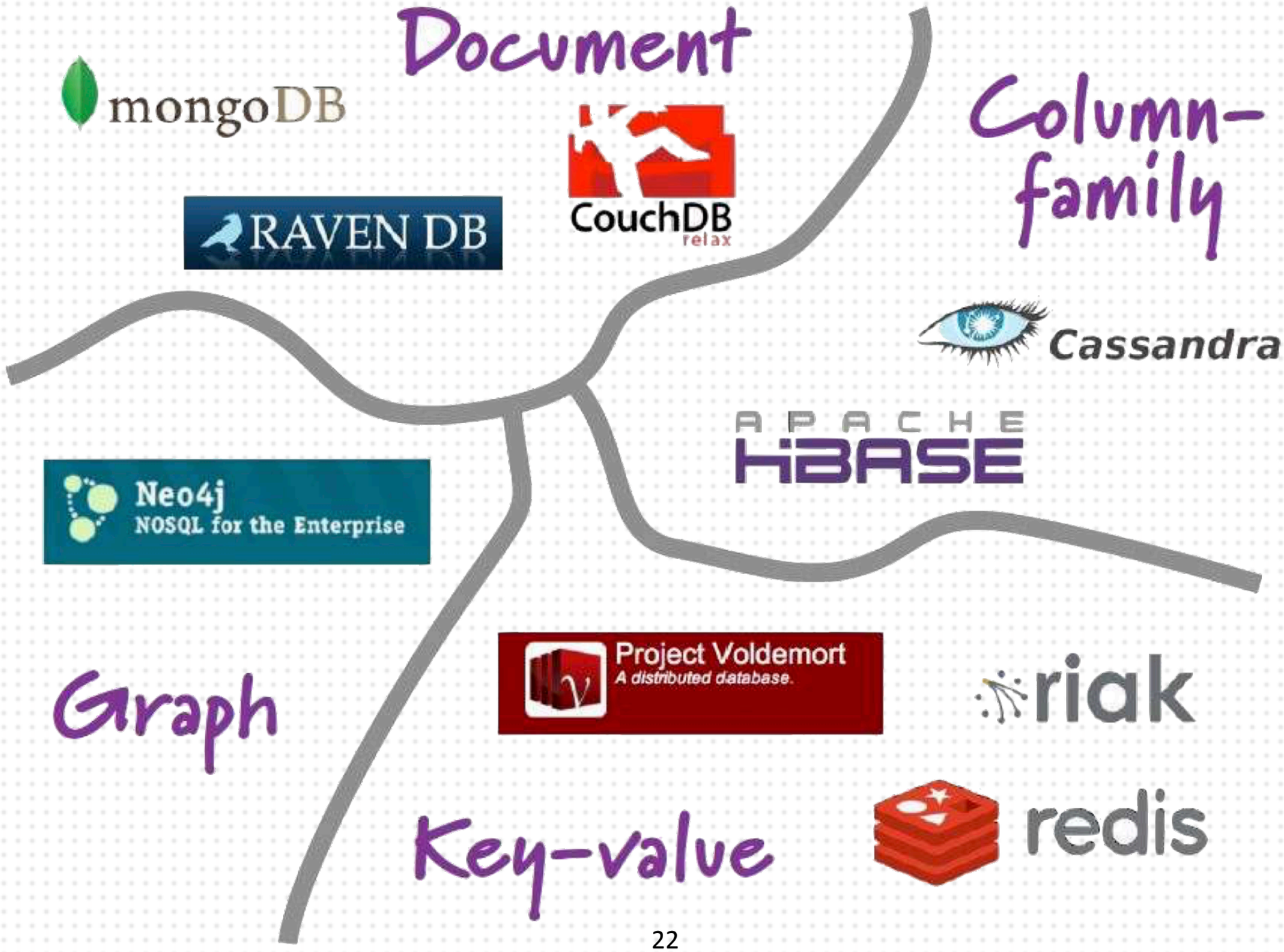








Four Major Aggregate Data Models



Key-Value Data Model

- The main idea is the use of a hash table
- Access data (values) by strings called keys
- Data has no required format – data may have any format
- Data model: (key, value) pairs
- Basic Operations:
  - Insert(key,value), Fetch(key), Update(key), Delete(key)

Car	
Key	Attributes
1	Make: Nissan Model: Pathfinder Color: Green Year: 2003
2	Make: Nissan Model: Pathfinder Color: Blue Color: Green Year:2005 Transmission: Auto

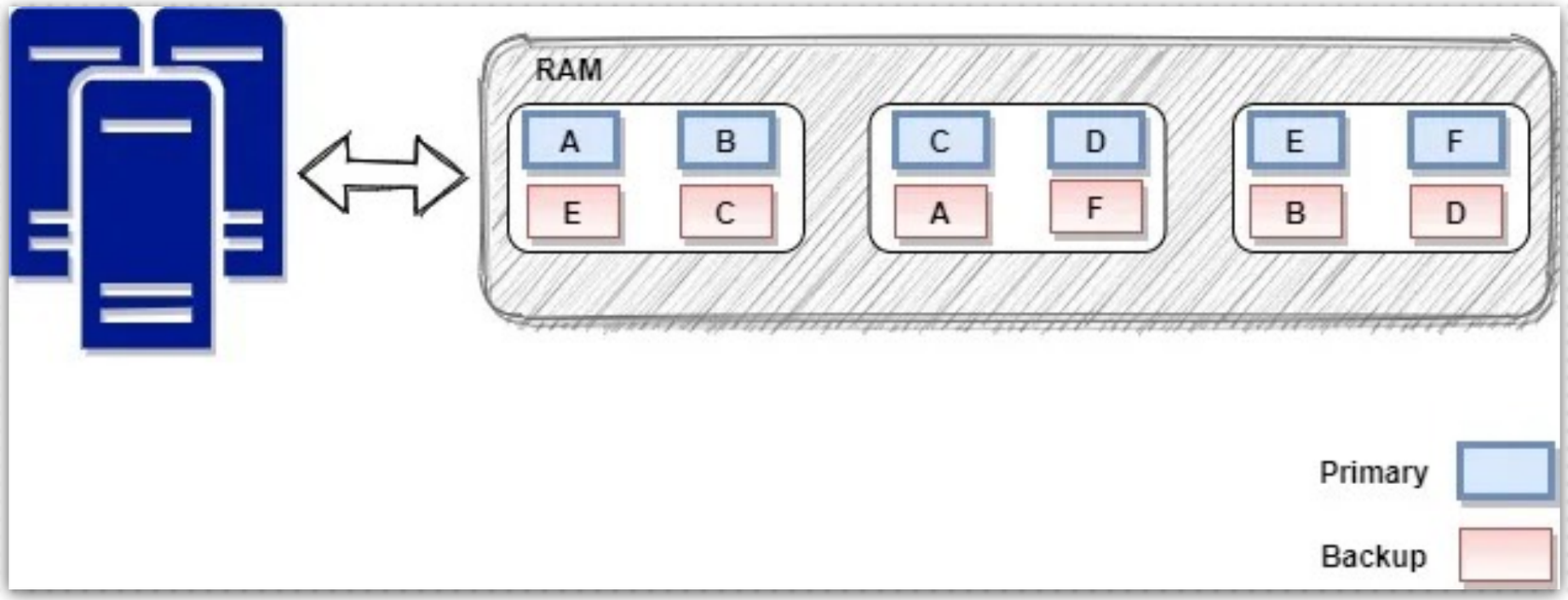
Key-Value Data Model

- Value is stored as a “blob”
  - Without caring or knowing what is inside
  - Application is responsible for understanding the data
- Main observation from Amazon (using Dynamo)
  - “There are many services on Amazon’s platform that only need primary-key access to a data store.”
  - E.g. Best seller lists, shopping carts, customer preferences, session management, sales rank, product catalog



Hazelcast

- Hazelcast IMDG is an open source in-memory data grid based on Java.
- A Data Grid is a system of multiple servers that work together to manage information and related operations in a distributed environment.



Using IMDG Results in:

- **Performance increase:** One of the best advantages of an In-Memory Data Grid is the increase in the entire system performance.
- **Data structure:** the data structure itself is a simple key-value store, rather than a complex relational model (still providing ACID properties)
- **Operations:** from an operational perspective, we already saw how an IMDG can be scaled up and down and how redundancy is built-in, which will allow us to easily achieve a highly-available system.

Connecting to the IMDG

- Connecting to two instances

```
Config config = new Config();
HazelcastInstance h1 = Hazelcast.newHazelcastInstance(config);
HazelcastInstance h2 = Hazelcast.newHazelcastInstance(config);
```

- Data Manipulation

```
import com.hazelcast.core.*; !
import java.util.ConcurrentMap;!

Map<String, Customer> map = hazelcast.getMap(“customers");
map.put ("1", customer);
Customer c = map.get("1");

//ConcurrentMap methods!
map.putIfAbsent ("2", new Customer("Chuck Norris”));
map.replace ("3", new Customer("Bruce Lee"));
```

Transactions

```
import com.hazelcast.core.*; import java.util.*;

TransactionOptions options = new
   TransactionOptions().setTransactionType(TransactionType.TWO_PHASE);
TransactionContext context = hz.newTransactionContext(options);
context.beginTransaction();

TransactionalQueue queue = context.getQueue(“myqueue");
TransactionalMap map = context.getMap(“mymap");
TransactionalSet set = context.getSet("myset");

try {
    Object obj = queue.poll();
    //process obj
    map.put ("1", "value1");
    set.add ("value");
    //do other things..
    context.commitTransaction();
}catch (Throwable t)  {
    context.rollbackTransaction();
}
```

- Java **MapStore** and **MapLoader** interfaces to implement a custom MapStore with your own logic, loading data from a data store, and writing data to a data store.

- MapLoader: The methods on the MapLoader interface get invoked when the application requests a value from the map
- MapStore: The methods on the MapStore interface can also replicate updates made to a map to the data store. This replication can be implemented blocking (write-through) or non-blocking (write-behind)

Column Family Data Model

- The column is lowest/smallest instance of data.
- It is a tuple that contains a name, a value and a timestamp

ColumnFamily: Authors

Key	Value
"Eric Long"	Columns Name: "email" — Value: "eric (at) long.com" Name: "country" — Value: "United Kingdom" Name: "registeredSince" — Value: "01/01/2002"
"John Steward"	Columns Name: "email" — Value: "john.steward (at) somedomain.com" Name: "country" — Value: "Australia" Name: "registeredSince" — Value: "01/01/2009"
"Ronald Mathies"	Columns Name: "email" — Value: "ronald (at) sodeso.nl" Name: "country" — Value: "Netherlands, The" Name: "registeredSince" — Value: "01/01/2010"

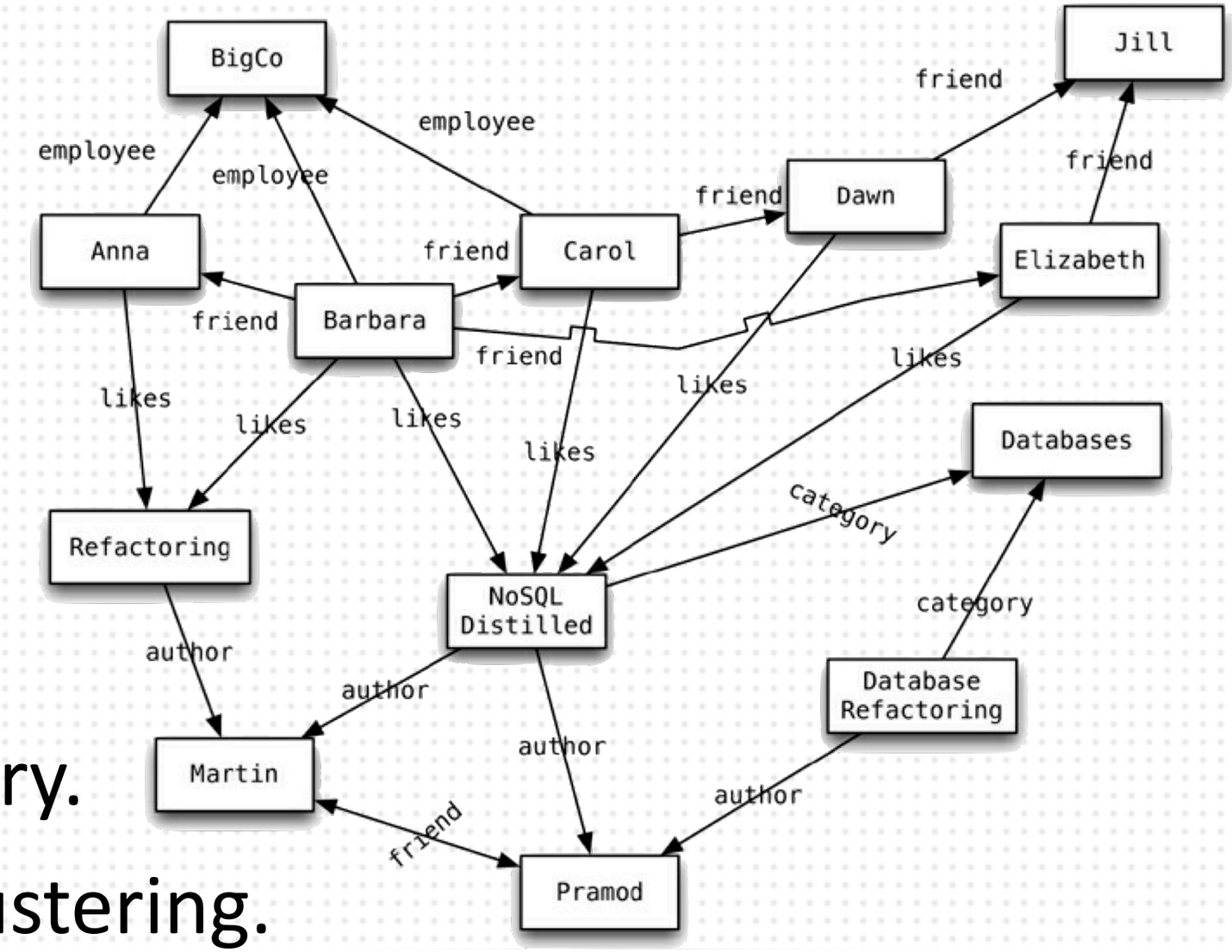
Column Family Data Model

Some statistics about Facebook Search (using Cassandra)

- MySQL > 50 GB Data
  - Writes Average : ~300 ms
  - Reads Average : ~350 ms
- Rewritten with Cassandra > 50 GB Data
  - Writes Average : 0.12 ms
  - Reads Average : 15 ms



Graph Data Model



- Based on Graph Theory.
- Scale vertically, no clustering.
- You can use graph algorithms easily
- Transactions ACID

Document-Based Data Model

- Usually JSON like interchange model.
- Query Model: JavaScript-like or custom.
- Aggregations: Map/Reduce
- Indexes are done via B-Trees.
- Unlike simple key-value stores, both keys and values are fully searchable in document databases.



Connecting to MongoDB

- Very similar to connecting to RDBMS

```
MongoClient mongoClient = MongoClients.create(“user:pass//localhost:27017");
```

```
MongoClient mongoClient = new MongoClient("db1.server.com", 27018);
```

- But there is an easy to use option for connecting to a cluster of servers

```
MongoClient mongoClient = MongoClients.create(
    MongoClientSettings.builder()
        .applyToClusterSettings(builder ->
            builder.hosts(Arrays.asList(
                new ServerAddress("host1", 27017),
                new ServerAddress("host2", 27017))))
    .build()
);
```

Connecting to MongoDB

- Databases and Collections

```
//database instance
MongoDatabase database = mongoClient.getDatabase("test");
//collection instance
MongoCollection<Document> collection = database.getCollection("col1");
```

- Security

```
MongoClient mongoClient = new MongoClient();
MongoDatabase db = mongoClient.getDatabase("test");
char[] password = new char[] {'s', 'e', 'c', 'r', 'e', 't'};
boolean authenticated = db.authenticate("root", password);

if (authenticated) {
    System.out.println("Successfully logged in to MongoDB!");
} else {
    System.out.println("Invalid username/password");
}
```

Collections Manipulation

- Creating a collection

```
database.createCollection("cappedCollection",
            new CreateCollectionOptions().capped(true).sizeInBytes(0x100000));
```

- Define document validations

```
ValidationOptions collOptions = new ValidationOptions().validator(
        Filters.or(Filters.exists("email"), Filters.exists("phone")));
database.createCollection("contacts",
        new CreateCollectionOptions().validationOptions(collOptions));
```

- Drop a collection

```
MongoCollection<Document> collection = database.getCollection("contacts");
collection.drop();
```

Defining Indexes

- Ascending, descending, and compound indexes

```
collection.createIndex(Indexes.ascending("name"));

collection.createIndex(Indexes.ascending("stars", "name"));

collection.createIndex(Indexes.descending("stars", "name"));

collection.createIndex(Indexes.compoundIndex(Indexes.descending("stars"),
Indexes.ascending("name")));
```

- Unique and partial indexes (only index valid documents)

```
IndexOptions indexOptions = new IndexOptions().unique(true);
collection.createIndex(Indexes.ascending("name", "stars"), indexOptions);

IndexOptions partialFilterIndexOptions = new IndexOptions()
          .partialFilterExpression(Filters.exists("contact.email"));
collection.createIndex(
          Indexes.descending("name", "stars"), partialFilterIndexOptions);
```

Different Types of Indexes

- Traditional text and hashed indexes

```
collection.createIndex(Indexes.text(“name"));

collection.createIndex(Indexes.hashed("_id"));
```

- Location-Based indexes which their efficiencies is based on the nature of data and queries

```
collection.createIndex(Indexes.geo2dsphere("contact.location"));

collection.createIndex(Indexes.geo2d("contact.location"));

IndexOptions haystackOption = new IndexOptions().bucketSize(1.0);
collection.createIndex(
        Indexes.geoHaystack("contact.location", Indexes.ascending("stars")),
        haystackOption);
```

Query a Collection

- Finding documents by passing a filter as a criteria

```
FindIterable<Document> result = collection.find(eq("name", "456 Cookies Shop"));
```

- More complex criteria: the stars field is greater than or equal to 2 and less than 5, AND the categories field equals "Bakery" (or if categories is an array, contains the string "Bakery" as an element):

```
collection.find(
    new Document("stars", new Document("$gte", 2)
          .append("$lt", 5))
          .append("categories", "Bakery"));
```

```
import static com.mongodb.client.model.Filters.*;

collection.find(and(gte("stars", 2), lt("stars", 5),
        eq("categories", “Bakery")));
```

Projection

- Queries in MongoDB return all fields in matching documents.
- Specify a projection document to return the needed fields
- The following includes a projection Document which specifies that the matching documents return only the name field, stars field, and the categories field.

```
import static com.mongodb.client.model.Projections.*;

collection.find(and(gte("stars", 2), lt("stars", 5), eq(“categories",
      “Bakery"))
).projection(fields(include("name", "stars", "categories"), excludeId()));
```

Iterating the Result

- Sorting the result

```
collection.find(and(gte("stars", 2), lt("stars", 5), eq("categories",
     “Bakery"))
).sort(Sorts.ascending(“name"));
```

- MongoIterable: iterator(), first(), forEach(), map(), into()

```
Block<Document> printBlock = new Block<Document>() {
       @Override
       public void apply(final Document document) {
           System.out.println(document.toJson());
       }
};


collection.find(and(gte("stars", 2), lt("stars", 5), eq("categories",
     “Bakery"))
).sort(Sorts.ascending(“name”)).forEach(printBlock);
```

Writing to a Collection

- Inserting a document in a collection

```
Document document = new Document("name", "Café Con Leche")
      .append("contact", new Document("phone", "228-555-0149")
            .append("email", "cafeconleche@example.com")
            .append("location", Arrays.asList(-73.92502, 40.8279556))
      )
      .append("stars", 3)
      .append("categories", Arrays.asList("Bakery", "Coffee", "Pastries")
);
collection.insertOne(document);
```

- Inserting a set of documents

```
Document doc1 = ...
Document doc2 = ...

List<Document> documents = new ArrayList<Document>();
documents.add(doc1);
documents.add(doc2);

collection.insertMany(documents);
```

Updating Documents

- updateOne(): updates at most a single document, even if multiple documents can be matched
- Updating the restaurants collection whose _id is ObjectId("57506d62f57802807471dd41")
  - using “set” to set the value of stars to 1 and contact.phone to "228-555-9999"
  - using “currentDate” to modify the lastModified field to the current date.

```
collection.updateOne(
      eq("_id", new ObjectId("57506d62f57802807471dd41")),
      combine(set("stars", 1), set("contact.phone", "228-555-9999"),
      currentDate(“lastModified"))
);
```

Updating Documents

- updateMany(): updates at most a single document, even if multiple documents can be matched

```
collection.updateMany(
        eq("stars", 2),
        combine(set("stars", 0), currentDate(“lastModified"))
);
```

- Updating options
  - upsert: insert if there is no match
  - bypassDocumentValidation: ignore validation

```
collection.updateOne(
        eq("_id", 1),
        combine(set("name", "Tulips"), currentDate("lastModified")),
        new UpdateOptions().upsert(true).bypassDocumentValidation(true));
```

Replacing Document

- ReplaceOne(): replaces the entire document - fields in the old document not contained in the new will be lost

```
collection.replaceOne(
      eq("_id", new ObjectId("57506d62f57802807471dd41")),
      new Document("name", "Green Salads Buffet")
          .append("contact", "TBD")
          .append("categories", Arrays.asList("Salads", "Health
                Foods", “Buffet")
          )
);
```

- You are allowed to update options(i.e. upsert and bypassDocumentValidation)

Delete Documents

- Delete one document

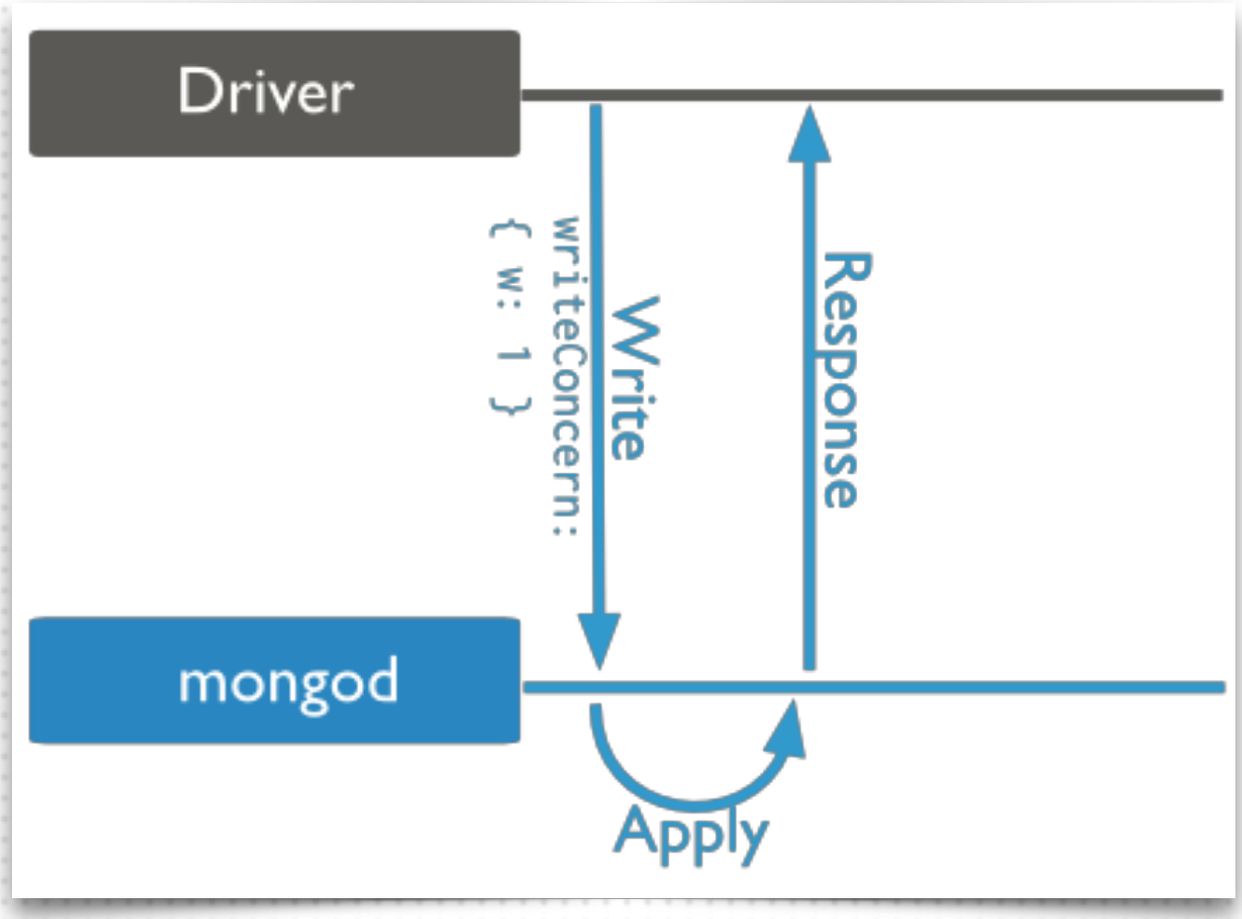
```
collection.deleteOne(eq("_id", new ObjectId("57506d62f57802807471dd41")));
```

- Delete many documents

```
collection.deleteMany(eq("stars", 4));
```

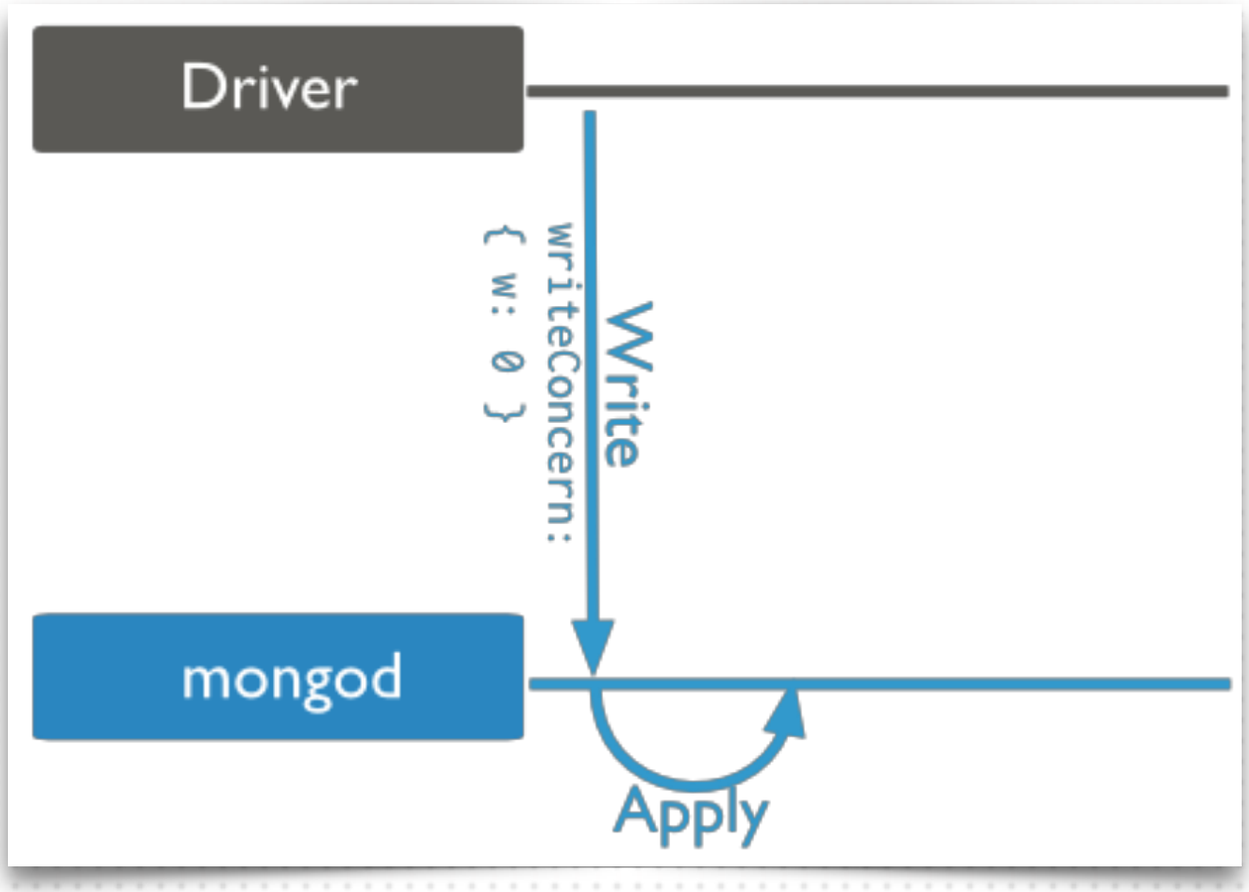
Write Concerns

- WriteConcern.ACKNOWLEDGED



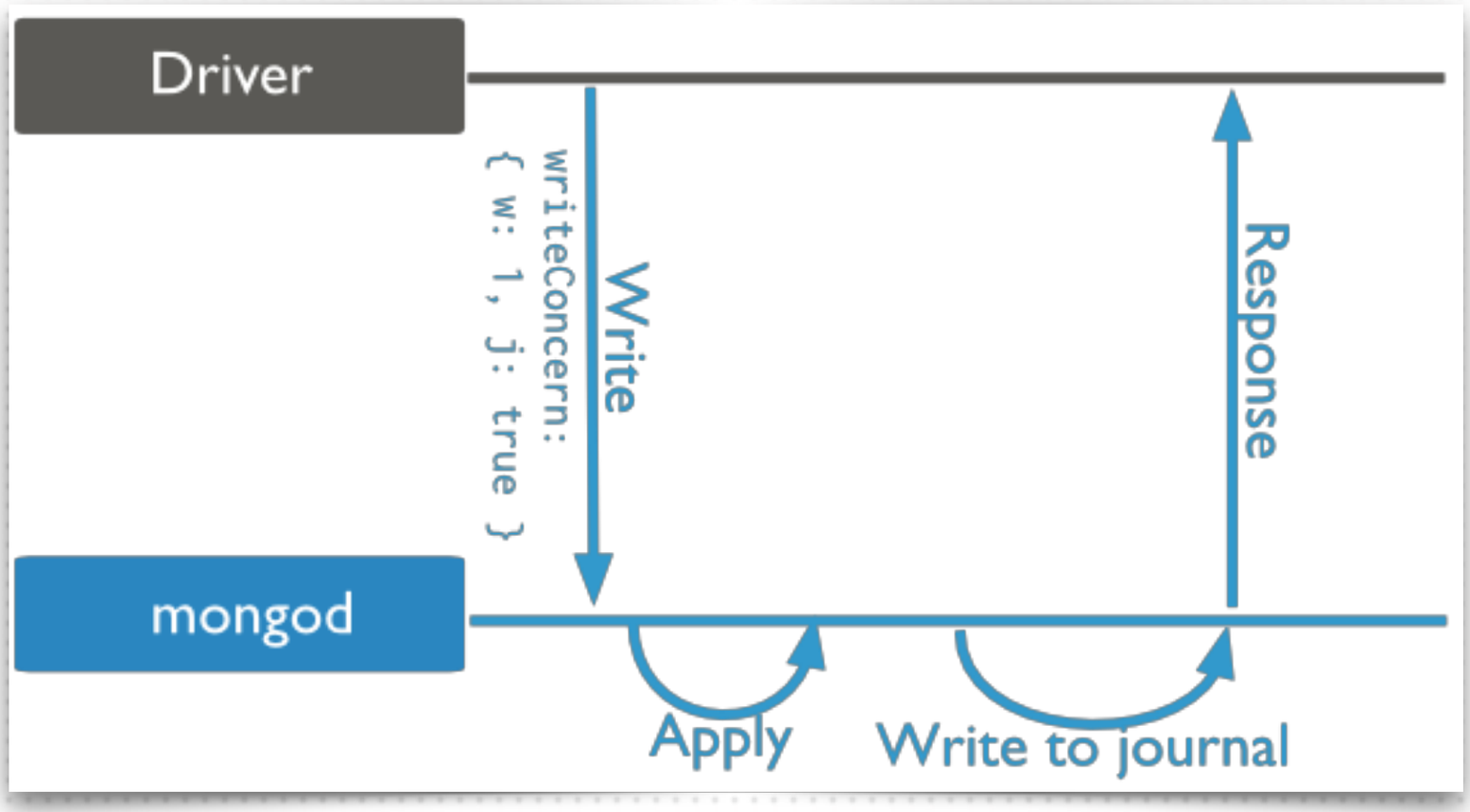
Write Concerns

- WriteConcern.ACKNOWLEDGED
- WriteConcern.UNACKNOWLEDGED



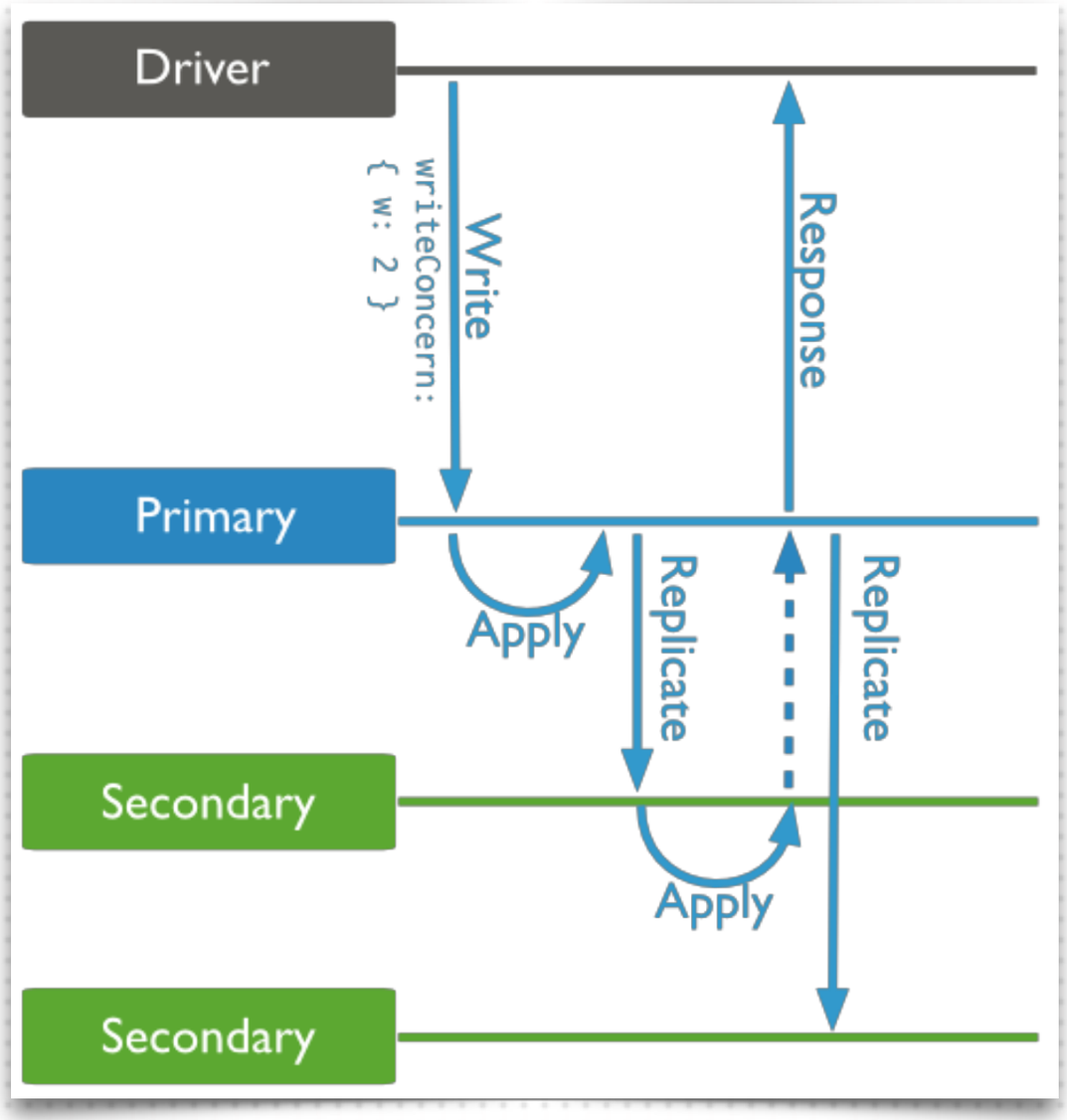
Write Concerns

- WriteConcern.ACKNOWLEDGED
- WriteConcern.UNACKNOWLEDGED
- WriteConcern.JOURNALED



Write Concerns

- WriteConcern.ACKNOWLEDGED
- WriteConcern.UNACKNOWLEDGED
- WriteConcern.JOURNALED
- WriteConcern.
  REPLICA_ACKNOWLEDGED



Write Concerns

- WriteConcern.ACKNOWLEDGED
- WriteConcern.UNACKNOWLEDGED
- WriteConcern.JOURNALED
- WriteConcern.REPLICA_ACKNOWLEDGED
- WriteConcern.MAJORITY;

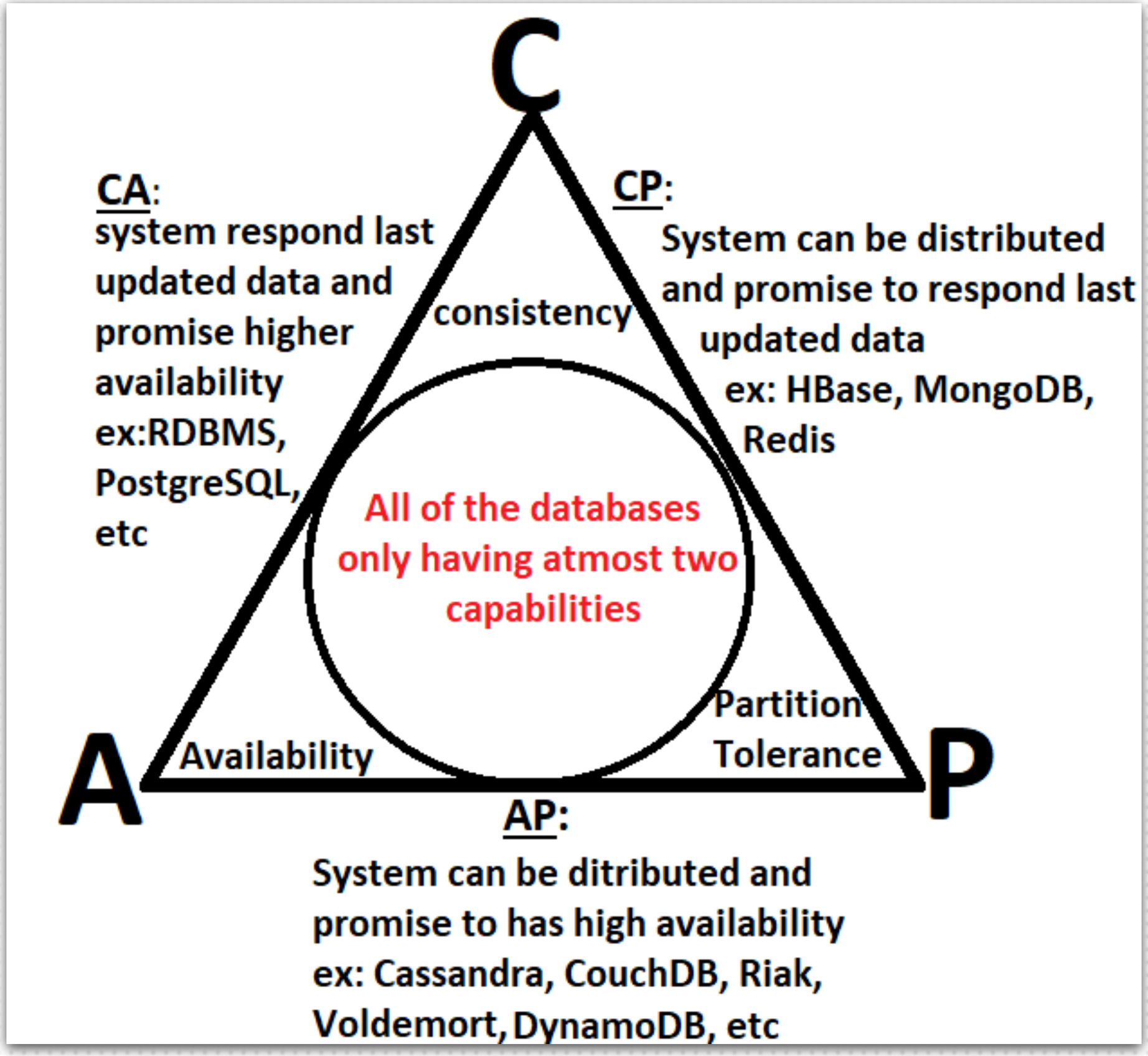
```
MongoClientOptions options =
    MongoClientOptions.builder().writeConcern(WriteConcern.MAJORITY).build();
MongoClient mongoClient = new MongoClient(Arrays.asList(
    new ServerAddress("host1", 27017),
    new ServerAddress("host1", 27018)), options
);
```

What We Need?

- We need a distributed database system having such features:
  - Fault tolerance
  - High availability
  - Consistency
  - Scalability
- Which is impossible!!! According to CAP theorem

CAP Theorem

- This theorem explains that it is not possible to guarantee three properties, consistency, availability, and partition tolerance at the same time in a distributed system with data replication
- **Consistency:** Every read receives the most recent write or an error among replicated copies.
- **Availability:** Every request receives a (non-error) response, without the guarantee for containing the most recent write.
- **Partition tolerance:** The system continues to operate in the case that nodes in the system being partitioned by a network fault.


C
CA:
system respond last updated data and promise higher availability
ex:RDBMS, PostgreSQL, etc
consistency
CP:
System can be distributed and promise to respond last updated data
ex: HBase, MongoDB, Redis
All of the databases only having atmost two capabilities
A
Availability
Partition Tolerance
P
AP:
System can be ditributed and promise to has high availability
ex: Cassandra, CouchDB, Riak, Voldemort, DynamoDB, etc

Thinking Differently

- In some cases getting an answer quickly is more important than getting a correct answer
- By giving up ACID properties, one can achieve higher performance and scalability.
- Any data store can achieve Atomicity, Isolation and Durability but do you always need consistency?
- Maybe we should implement Asynchronous Inserts and updates and should not wait for confirmation?

BASE

- Almost the opposite of ACID.
- **Ba**sically available: Nodes in the a distributed environment can go down, but the whole system shouldn’t be affected.
- **S**oft State (scalable): The state of the system and data changes over time.
- **E**ventual Consistency: Given enough time, data will be consistent across the distributed system.

ACID vs. BASE

- ACID
  - Strong consistency
  - Less availability
  - Pessimistic concurrency
  - Complex
- BASE
  - Availability is the most important thing. Willing to sacrifice for this (CAP)
  - Weaker consistency (Eventual)
  - Best effort
  - Simple and Fast
  - Optimistic

PolyglotPersistence

